

WS2815 16×14 LED Matrix Troubleshooting & Setup Guide

Practical Step-by-Step Hardware Debugging & Production Software Guide | Arduino Uno R3 & Linux Pi

Matrix Layout: 16 vertical strips of 14 LEDs each (224 LEDs total), connected in a vertical serpentine (zigzag) chain starting at the top-left.
Power & Ground Rule: Always connect the **Ground (GND)** of your 12V supply to the **GND pin of your Arduino**. Without a shared ground, signals float and glitch. Software brightness defaults to a safe 20% to prevent excess heat and power draw.

PART 1: STEP-BY-STEP HARDWARE TROUBLESHOOTING

Step 1: Inspect Physical Wiring & Solder Joints (Power Disconnected)

Before powering up, verify these connections with your eyes and a multimeter in continuity (beep) mode:

- **First LED Data Input ('Needs DI not BI'):** The first LED has four pads: +12V, DI, BI, GND. Make sure Arduino Pin D6 connects to **DI** (through your 330Ω resistor). The **BI** pad on this first LED must be connected directly to **GND**. If BI is left disconnected, electrical noise can cause the first pixel to randomly trigger its backup mode.
- **Strip-to-Strip Joins:** Each strip connects to the next via: Data Out (DO) → Data In (DI), Backup Out (BO) → Backup In (BI), +12V → +12V, and GND → GND. Verify DO and BO were not swapped.
- **Multimeter Check for Solder Bridges:** If two LEDs previously lit up simultaneously, examine that junction closely. Probe with your multimeter between: (1) **DI and DO** (must NOT beep), and (2) **DI and BI** (must NOT beep). If either beeps, you have a physical solder bridge joining pads together.

Step 2: Run the Standalone Uno Chase Test (Decisive Hardware Check)

This test runs completely on the Arduino. It does not use Python, USB serial, or coordinate mapping. It lights exactly **one LED at a time** from index 0 to 223, clearing all others on every step.

1. Open `uno_diagnostic_chase/uno_diagnostic_chase.ino` in the Arduino IDE.
 2. Select **Board: Arduino Uno** and click **Upload**. Open the Serial Monitor at **115200 baud**.
- **Expected Visual Pattern:** Index 0 (Top-Left) lights **White**. A single **Cyan** light steps down strip 0 to the bottom (index 13, **Red**). Then index 14 (bottom of strip 1) lights **Green**, and steps up to index 27 (Red).
 - **How to Interpret What You See:**
 - **Case A (Two adjacent LEDs light at once):** If two LEDs still light together, this **proves a hardware defect** (a solder bridge between DI and DO/BI on the strip PCB, or a bad LED chip). Software cannot fix this. Clean the junction pads with solder wick, then re-test.
 - **Case B (Only one LED lights at a time through all 224 LEDs):** Your hardware and wiring are **100% physically sound!** Any previous double-lighting was just a software artifact. Proceed to Step 3.

Step 3: Understanding the Corner & Text Distortion Bug

- **The Clue:** Previously, a corner test lit white 'one row above the bottom on the right edge' (index 208, or 15 LEDs from the end), and scrolling text was badly distorted.
- **What Caused It:** Your physical matrix is made of 16 *vertical* strips (14 LEDs each). When the old software calculated positions assuming *horizontal* strips, the bottom-right corner (col 15, row 13) calculated as index 208. On vertical strips, index 208 physically sits at column 14, row 12 (exactly one row above the bottom!). Because rows and columns were flipped, text was sliced and unreadable.
- **The Fix:** The new `matrix_controller.py` uses the exact **Vertical Serpentine** coordinate formula. The corners and letters now map correctly.

Step 4: Understanding the 'b'N" (Not Acknowledged) Error

- **What Caused It:** While the Arduino Uno is updating WS2815 LEDs (~7 ms), it must temporarily turn off its internal interrupts. At 115,200 baud, if the Linux computer kept sending data during this time, the Arduino's tiny 64-byte serial buffer overflowed, dropping bytes and causing checksum failures ('b'N').
- **The Fix:** The new programs use a strict 'take-turns' rule (half-duplex handshake). The host sends 1 frame and waits. The Arduino displays the frame, and only then replies with 'K' (ACK). The computer never sends while the Arduino is busy.

PART 2: COMPLETE SETUP & OPERATING GUIDE

Step 1: Upload the Production Firmware to Arduino

1. In Arduino IDE, open uno_matrix_receiver/uno_matrix_receiver.ino. Make sure **FastLED** is installed (Tools → Manage Libraries).
2. Select **Board: Arduino Uno** and your Port, then click **Upload**.
3. On boot, all LEDs clear to black, and the Uno sends READY\n. It is now waiting for frames from your Linux Pi.

Step 2: Set Up Your Linux Computer (Raspberry Pi / Orange Pi 5)

Connect the Arduino to your Linux board via USB. Open a terminal and run:

```
# 1. Install pyserial library:
pip install pyserial
# 2. Grant your user account access to the USB serial port:
sudo usermod -a -G dialout $USER
# (Log out and log back in once if you had to add the group)
```

Step 3: Run Verification Tests in Order

Test Name	Command to Run	Expected Result on Real Panel
1. Corner Test	python3 matrix_controller.py corners	Top-Left: Red Top-Right: Green Bottom-Left: Blue Bottom-Right: White
2. Strip Scan	python3 matrix_controller.py grid --mode col --delay 0.3	Lights vertical columns one by one from left to right (strip 0 to 15).
3. Row Scan	python3 matrix_controller.py grid --mode row --delay 0.3	Lights horizontal rows one by one from top (row 0) to bottom (row 13).

Step 4: Displaying Text & Operating Commands Cheat Sheet

Feature / Goal	Command Line Instruction	Description & Options
Scroll Bold Text	python3 matrix_controller.py text 'HELLO' --scale 2 --speed 14	Full-height 10×14 font. Smooth horizontal scroll across all 14 rows.
Stationary Text	python3 matrix_controller.py text 'HI' --scale 2 --stationary	Displays centered stationary text without scrolling.
Custom Color	python3 matrix_controller.py text 'OPEN' --color 0,255,100	Format is R,G,B (e.g. 255,0,0=Red, 0,255,0=Green, 255,180,0=Amber).
Safe Brightness	python3 matrix_controller.py --brightness 30 text 'TEST'	Percentage 1..100. Default is 20% for cool, safe operation.
Terminal Preview (No LEDs needed!)	python3 matrix_controller.py --no-serial --preview text 'OK' --scale 2 --stationary	Draws the exact 16×14 grid in full 24-bit color directly on your computer screen!
Clear Matrix	python3 matrix_controller.py clear	Instantly blanks out all 224 LEDs.

PART 3: QUICK SYMPTOM & FIX CHEAT SHEET

Symptom Observed	Likely Root Cause	Actionable Solution
First LED flickers or matrix ignores data	Data connected to BI instead of DI, or BI is floating	Connect Uno D6 (via 330Ω) to DI . Solder the first LED's BI to GND .
Two adjacent LEDs stay lit together during chase	Solder bridge across DI/DO or DI/BI at that junction	Clean the copper pads with solder wick or a clean soldering iron tip.
Chase stops completely at a certain strip	Broken wire, cold solder joint, or disconnected 12V/GND	Re-flow solder at the joint between the working strip and the dark strip.
Python: 'Permission denied: /dev/ttyACM0'	Linux account missing serial permissions	Run sudo usermod -a -G dialout \$USER, then log out and log back in.
Panel turns black after 5s of closing Python	Built-in Auto-Blackout safety feature	Normal operation: Arduino turns off lights if communication disconnects.